Documentation of original Inventory management system

Introduction:

This is a document of a system that stores data and generates bills to the customer.

This document includes usage, tools, architecture, classes and what they consist of, and code section that goes over each function in each file.

There is another file called changelog where any changes to these files are documented.

Usage:

Start up

- Create new .venv (I had to since original was broken according to visual code)
- install pil to the venv
- run database.py.
- run dashboard.py and run billing.py on different python instances.

After this you can create, edit, and remove employees, suppliers, products, and categories in their respective views that can be opened via the dashboard. Billing can be accessed with the billing.py where bills can be generated and exported as sales. Billing view allows basic arithmetic calculation to be performed with in built calculator. Sales can be seen on sales view opened from dashboard.

Tools:

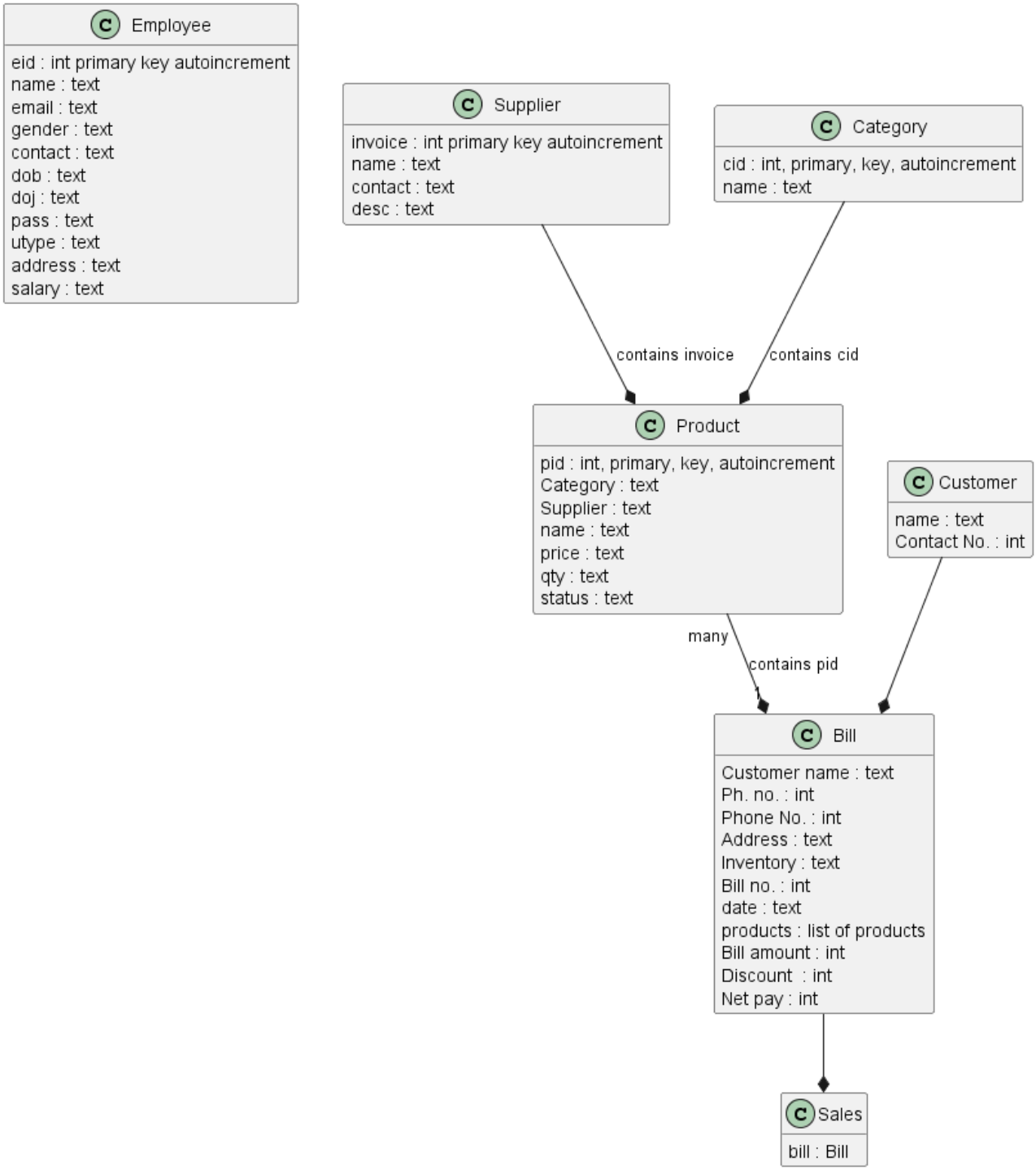
System uses python (python 3 recommended) and its libraries sqlite3, OS, tkinter, and pil.

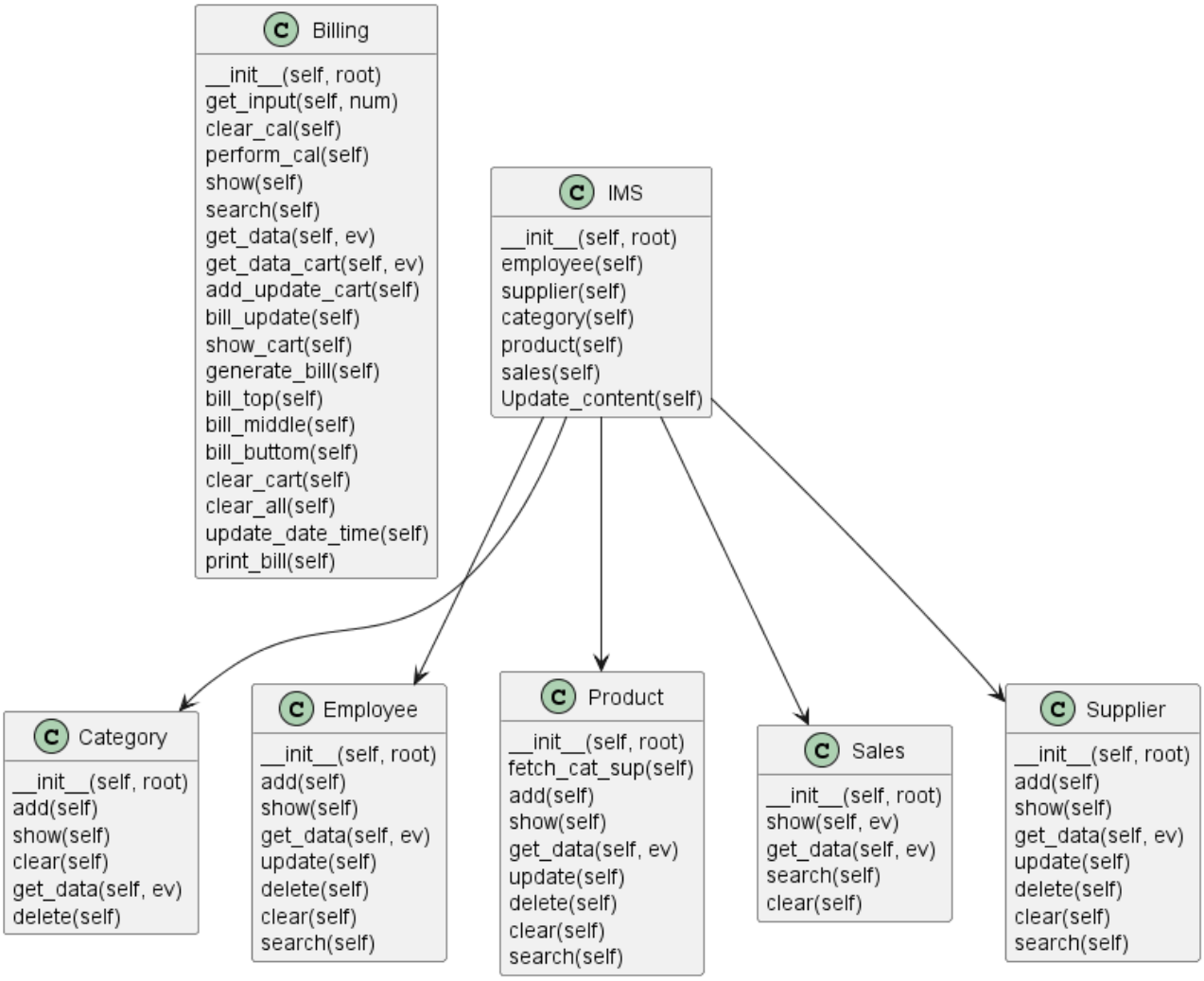
Architecture:

System’s code is divided into 8 files. Seven code files are to control their corresponding view and one is to simply generated tables for the system’s database. These 7 files code files are large single class files that include all necessary information of that one view. Deeper analysis is done in code section.

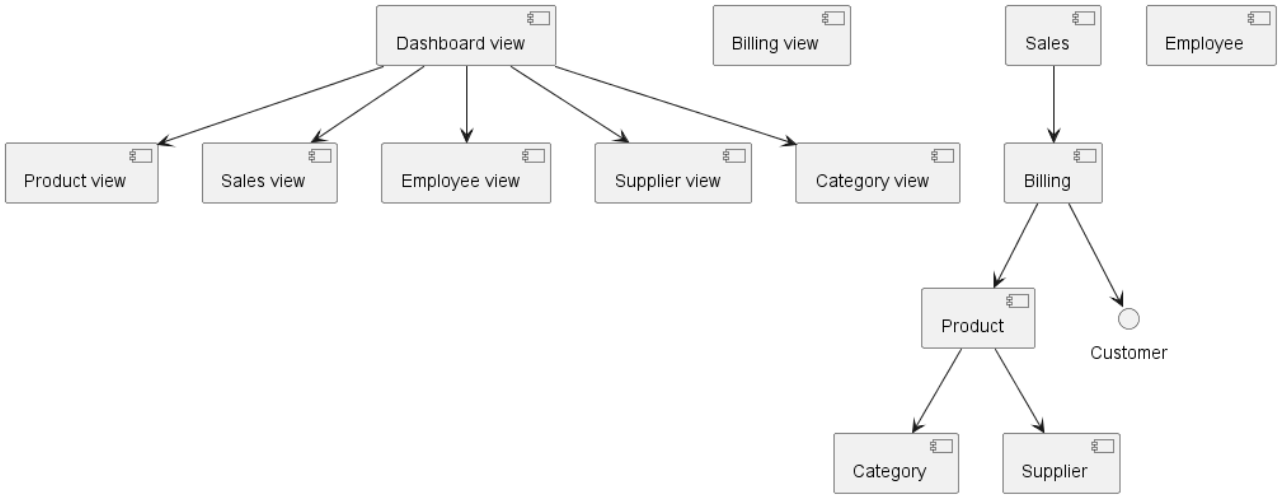
The system uses 1 database file, 7 image files that are on highest level in directory hierarchy, image folder that contains 8 image files, and bill folder that contains created sales/bills.

Class diagrams:


Employee
eid : int primary key autoincrement
name : text
email : text
gender : text
contact : text
dob : text
doj : text
pass : text
utype : text
address : text
salary : text
Supplier
invoice : int primary key autoincrement
name : text
contact : text
desc : text
Category
cid : int, primary, key, autoincrement
name : text
contains invoice
contains cid
Product
pid : int, primary, key, autoincrement
Category : text
Supplier : text
name : text
price : text
qty : text
status : text
Customer
name : text
Contact No. : int
many
contains pid
Bill
Customer name : text
Ph. no. : int
Phone No. : int
Address : text
Inventory : text
Bill no. : int
date : text
products : list of products
Bill amount : int
Discount : int
Net pay : int
Sales
bill : Bill

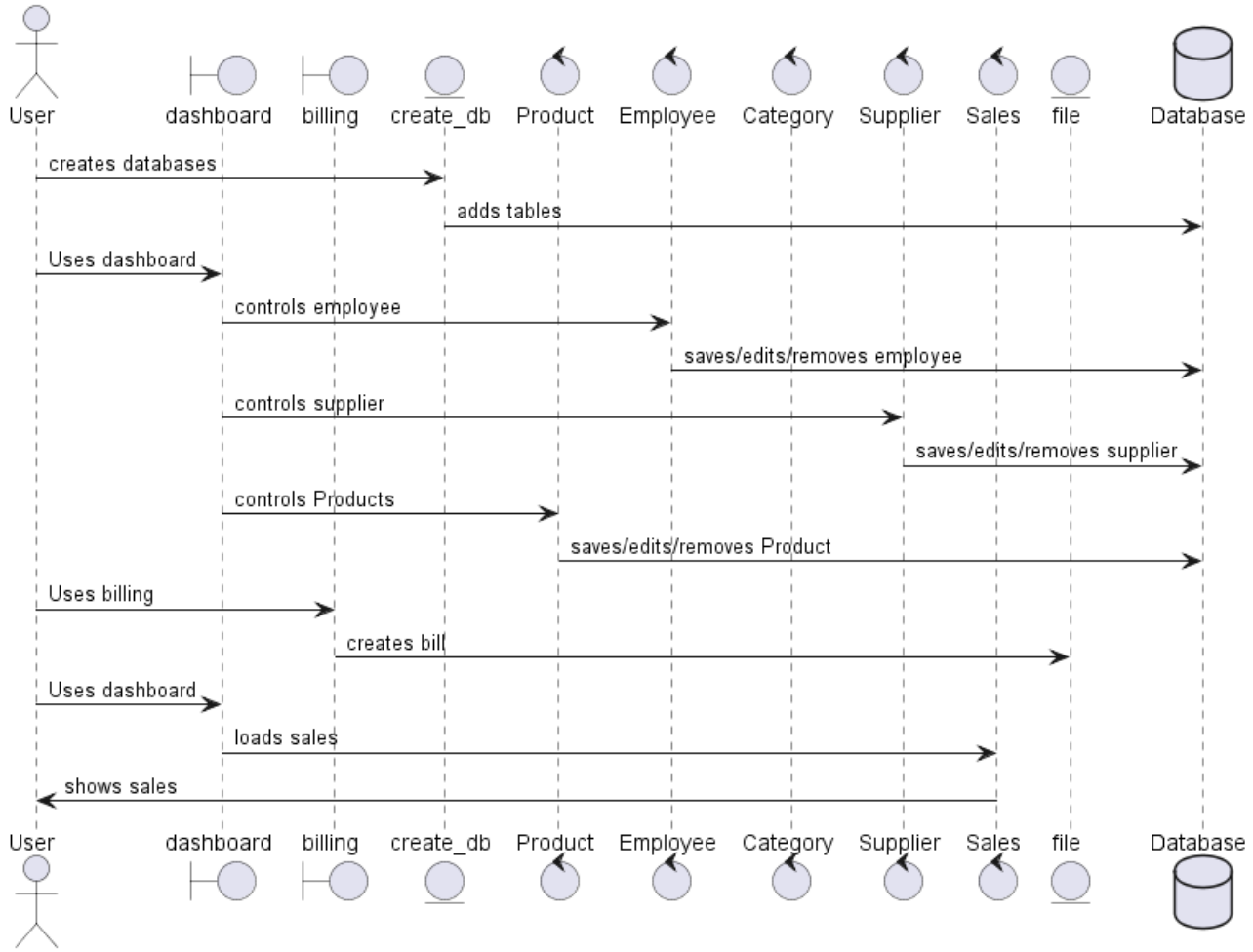


Dependency Diagram



On left shows views dependencies and on right object dependencies.

Sequence Diagram



Visualizes README.md actions.

Classes:

Employee (in database):

- eid (int, primary, key, auto-increment)
- name (text)
- email (text)
- gender (text)
- contact (text)
- dob (text)
- doj (text)
- pass (text)
- utype (text)
- address (text)
- salary (text)

Supplier (in database):

- invoice (int, primary, key, auto-increment)
- name (text)
- contact (text)
- desc (text)

Category (in database):

- cid (int, primary, key, auto-increment)
- name (text)

Product (in database):

- pid (int, primary, key, auto-increment)
- Category (text)
- Supplier (text)
- name (text)
- price (text)
- qty (text)
- status (text)

Customer (stored in bill, kind of)

- name (text)
- Contact No. (int)

Bill (stored in sales)

- Customer name (text)
- Ph. no. (customer No.) (int)
- Phone No. (int)
- Address (text)
- Inventory (text)
- Bill no. (int)
- date (text)
- products (list of products)
- Bill amount (int)
- Discount (int)
- Net pay (int)

Sales (txt file)

- Bill (object)

Code:

billing.py:

- lines 434-437: creates Tkinter's root, the billClass, and starts billClasses' main loop

In billClass:

- __init__(self, root):
  - lines 11-16: sets basic parameters for the screen, like size, is it resizeable, background color, cart_list where all the bill items are stored, and check print value that indicates if a bill has been generated.
  - "Title" sets window name and icon.
  - "Logout button" is a big yellow button without function (command parameter) at fixed location top right of the screen. It doesn't do anything.
  - "Clock" is welcome message to the system that shows current time
  - "Product frame" is a product search and selection component where user can select what product are billed from customer
  - "Customer frame" is a component to input customer information for the billing
  - "Calculator frame" is a component is perform basic arithmetic calculations
  - "Cart frame" is a component that shows what items are currently being billed from customer
  - "Add cart widgets" is a component where items can be added, updated, and removed from the bill
  - "Billing area" is a box for the bill that is created by billing buttons component
  - "Billing buttons" is a set of buttons to finalize and generate the bill.
- get_input(self, num): Adds a number or operator to the calculator's text field.
- clear_cal(self): Empties calculator's text field.
- perform_cal(self): Performs calculator's calculation

- show(self): Connects to the database and fetches all products for product frame
- search(self): Connects to the database and fetches specified products for product frame
- get_data(self, ev): Takes one product's values from product frame to the customer frame
- get_data_cart(self, ev): Takes one product's values from cart frame to the customer frame
- add_update_cart(self): adds, updates, or removes an item from cart frame.
- bill_update(self): Updates values to billing buttons and cart frame
- show_cart(self): clears cart frame and adds values back.
- generate_bill(self): generates bill by calling other functions and writes end result
- bill_top(self): generates top part of the bill (heading)
- bill_middle(self): generates middle part of the bill (products)
- bill_buttom(self): generates lower part of the bill (total amount)
- clear_cart(self): clears cart frame
- clear_all(self): delete cart_list and empties all frames, except product frame.
- update_date_time(self): calls it self every 200ms to update software's time
- print_bill(self): turns bill into txt file and stores it.

Other notes:

- Everything is at fixed location
- Massive single class file that handles all GUI and functionality regarding to billing
- Some unused variables
- Multiple index out of range errors appear from usage

category.py:

- lines 132-135: creates Tkinter's root, the categoryClass, and starts categoryClass' main loop.
- Other notes:
  - Incorrect images path for cat.jpg

In categoryClass:

- __init__(self, root):
  - lines 8-12: Sets basic parameters to the window
  - "variables": creates to objects that have master, value, and name values.
  - "title": Adds title, category input components, and two buttons "add" and "delete"
  - "category details": adds box where all categories are shown
  - "images": loads and displays two images (image paths are incorrect)
- add(self): Adds category to the database and calls refreshment of "category details"
- show(self): clears "category details" and adds new categories back.
- clear(self): clears category input field and calls show()
- get_data(self, ev): collects data from item from "category details"
- delete(self): deletes a category

create_db.py:

- Creates tables to database if they do not exist yet.
- Calls create_db() by default when file is opened.

dashboard.py:

- lines 15-19: check if constant paths for image and bills directories exits and creates bill path if it does not exist.
- lines 230-233: creates Tkinter's root, the IMS, and starts IMS main loop.

in IMS (class):

- __init__(self, root):
  - lines 24-27 sets parameters for the window
  - "title": adds a logo and a title

    - "logout button": adds a useless button
    - "Clock": adds a clock
    - "left menu": adds an image and buttons to other views.
    - "content": adds boxes that show how many of each item type exist in the database
    - "footer": adds footer
- employee(self): functionality to a button to open employee view
- supplier(self): functionality to a button to open supplier view
- category(self): functionality to a button to open category view
- product(self): functionality to a button to open product view
- sales(self): functionality to a button to open sales view
- Update_content(self): updates content and clock values every 200ms

Other notes:
- There is a logout button, it doesn’t work
- Is it necessary to update all data every 200ms
- Imports and starts other views except billing view.

employee.py:
- lines 280-283: creates Tkinter's root, the employeeClass, and starts employeeClass' main loop.

in employeeClass
- __init__(self, root):
    - lines 8-13: set parameters for the window
    - "all variables": constructs a lot of stringVar objects
    - "search frame": creates title for employee search
    - "options": Creates search component.
    - "title": displayes text "Employee Details"
    - "Content": divided into rows: each row creates different part of input fields.
    - "buttons": creates four buttons "Save, "Update, "Delete", and "Clear"
    - "Employee details": Creates table where employees are shown.
- add(self): adds an employee to the database
- show(self): clears and shows adds employees
- get_data(self, ev): gets data from input fields.
- update(self): updates an employee
- delete(self): deletes an employee
- clear(self): empties the view's input boxes.
- search(self): searches employee.

product.py:
- lines 265-268: creates Tkinter's root, the productClass, and starts productClass' main loop.

In productClass:
- __init__(self, root):
    - lines 8-12: sets parameters for the windows.
    - "variables": creates class variables
    - "title": Creates title, label for input fields and input fields themselves on the left
    - "buttons": creates four buttons "Save, Update, Delete, Clear"
    - "Search frame": Creates search title
    - "options": creates input fields and button for search
    - "product details": Creates table for existing products.
- fetch_cat_sup(self): Fills cat_list with categories and sup_list with suppliers.
- add(self): adds adds product
- show(self): refreshes products table

- get_data(self, ev): gets data from input fields.
- update(self): Updates product
- delete(self): deletes product
- clear(self): clears input fields
- search(self): searches products

sales.py:
- lines 8-12: check if constant paths for image and bills directories exits and creates bill path if it does not exist.
- lines 137-140: creates Tkinter's root, the salesClass, and starts salesClass' main loop.

In salesClass:
- __init__(self, root):
  - lines: 17-24: sets parameters for the windows and class parameters.
  - "title": Creates title, invoice search field and two buttons "search and "clear".
  - "bill list": shows all existing bills
  - "bill area": show selected bill
  - "image": shows image "cat2.jpg", which does not have cat :(.
- show(self, ev): fetches and shows all bills
- get_data(self, ev): gets data from input fields
- search(self): searches bills
- clear(self): clears view.

supplier.py:
- lines 200-203: creates Tkinter's root, the supplierClass, and starts supplierClass' main loop.

In supplierClass:
- __init__(self, root):
  - lines: 8-12: sets parameters for the windows.
  - "all variables": creates class variables
  - "search frame": creates Invoice search.
  - "title": creates title
  - "content": has rows in it. Each row creates an input field to add, edit, or remove suppliers.
  - "buttons": creates four buttons "Save, "Update, "Delete", and "Clear"
  - "supplier details": creates table that shows suppliers.
- add(self): adds supplier
- show(self): refreshes suppliers list
- get_data(self, ev): reads input fields
- update(self): updates supplier
- delete(self): deletes supplier
- clear(self): empties datafields
- search(self): searches supplier